COMP 2035 – Operating Systems and Tool Chains (SP2 2022)

Course	Operating System and Tool Chains (COMP 2035)
Name	Gitae Bae
Email	baegy002@mymail.unisa.edu.au
Identifier	110310861
Date	Friday, 10/06/2022

Assignment 2 Report Operating System Design

[Gitae Bae]

[10/06/2022]

Contents

Scheduler ……3

Filesystem ……5

Memory Management ……8

Custom OS design ……10

References ……12

Scheduler

Scheduling for CPU is an OS(Operating System) behaviour technique that enables multiprogramming, assigning/disabling CPUs to processes in order, one of the key points in OS design, and also objective is to increase CPU utilization. When a process that was currently running transitions to another state, scheduling occurs if the next process is needed to select to run. Scheduling targets are scheduled only for processes in the staging list.

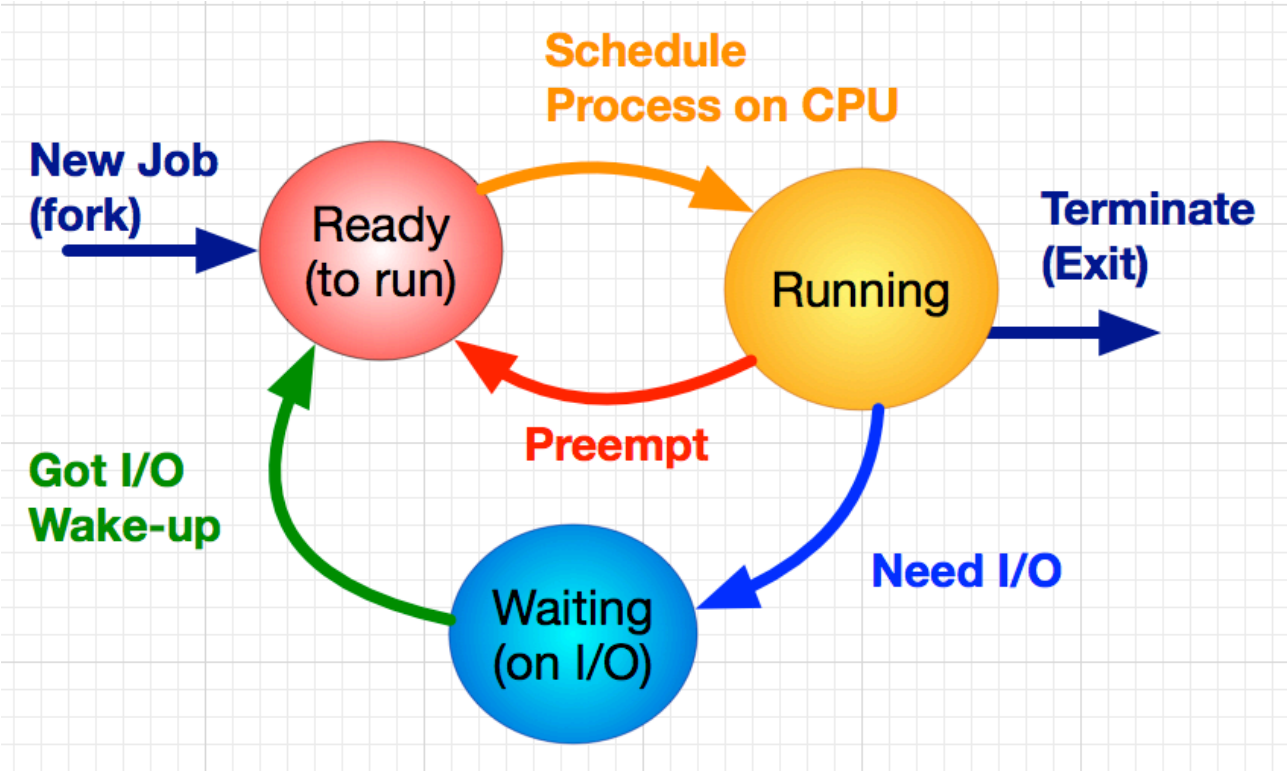

Figure 1: CPU Schedule Process (*The coordinator 2018*)

CPU scheduling is a process that permits one process to work the CPU while the implementation of another process is in a waiting state due to inaccessibility of any source like I/O, thereby making full work of the CPU (*StudyTonight 2022*). As shown below, it is CPU scheduling aims to make the system well.

·Seeking efficiency - Select a scheduler that maximizes CPU utilization and throughput and minimizes return time, latency, and response time.

·Seeking fairness - Select a scheduler by considering the optimal average value for each criterion and the minimum deviation between them.

·Interactive system - Select a scheduler with less variation in reaction time.

CPU scheduling has many algorithms. Among them, have a look at Multilevel Queue Scheduling and Multilevel Feedback Queue Scheduling. First of all, It is MultiLevel Queue Scheduling (MLQ).

·Form a staging queue for each priority.

·Always allocate CPUs to processes in the highest priority queue (preemptive scheduling that takes CPU away when a process arrives in a higher queue, even if a job is running in a lower priority queue).

·Each queue has its scheduling, such as RR (Round Robin) Scheduling or FCFS (First Come First Serve) Scheduling.

·Prioritize interactive, background, and other process characteristics.

·Less scheduling burden but less flexibility due to the inability to move processes between queues.

·Low priority processes may experience starvation waiting for CPU allocation for a long time.

The centre benefit of the multilevel queue is that it has a low scheduling overhead (*GeeksforGeeks 2022*). It has System Processes, Interactive Processes, Interactive Editing Processes, Batch Processes, and Student Processes. As shown in Figure 2, Processes are divided from highest to lowest priority.

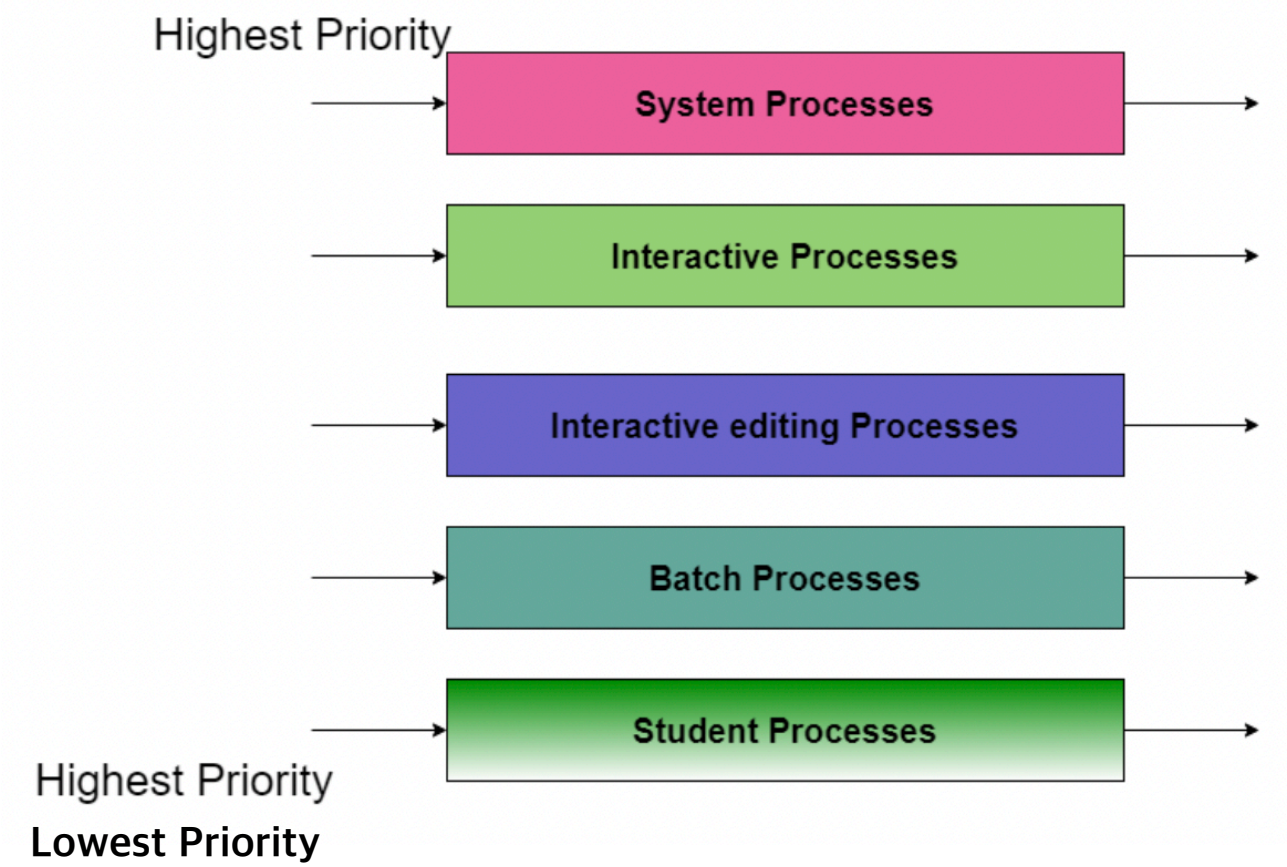

Figure 2: Processes of MultiLevel Queue Scheduling(MLQ) (*StudyTonight 2022*)

Secondly, there is Multilevel Feedback Queue Scheduling(MFQ).

·Multi-Step Queue + Apply Dynamic Process Priority Changes.

·When a process is created, it is registered in the highest priority preparation queue, and the registered process is executed with the CPU allocated in FCFS order. When the CPU time quantum for the corresponding queue is finished, it enters the preparation queue one step below.

·As the step goes down, the time quantum increases.

·Process can be moved between queues and CPU Burst is placed in a low priority queue and I/O Burst is placed in a high priority queue.

·The lowest queue is FCFS scheduling.

·If it waits too long in the bottom queue, it will go back to the top queue (prevent starvation through aging techniques).

This algorithm is important due to the low scheduling overhead. So, multi-level feedback queue scheduling requires complex judgments, such as the number of queues, algorithms for each queue, determining when to upgrade or downgrade priorities, and which queues the initial processes should enter. The biggest difference is that MLQ scheduling does not allow processes to move between queues, whereas MFQ scheduling allows processes to move between queues. Therefore, compared to MFQ scheduling, MLQ scheduling is less burdensome but less flexible. In addition, while MLQ scheduling may cause starvation due to the lack of CPU allocation in the lower queue, MFQ scheduling can prevent starvation through aging techniques.

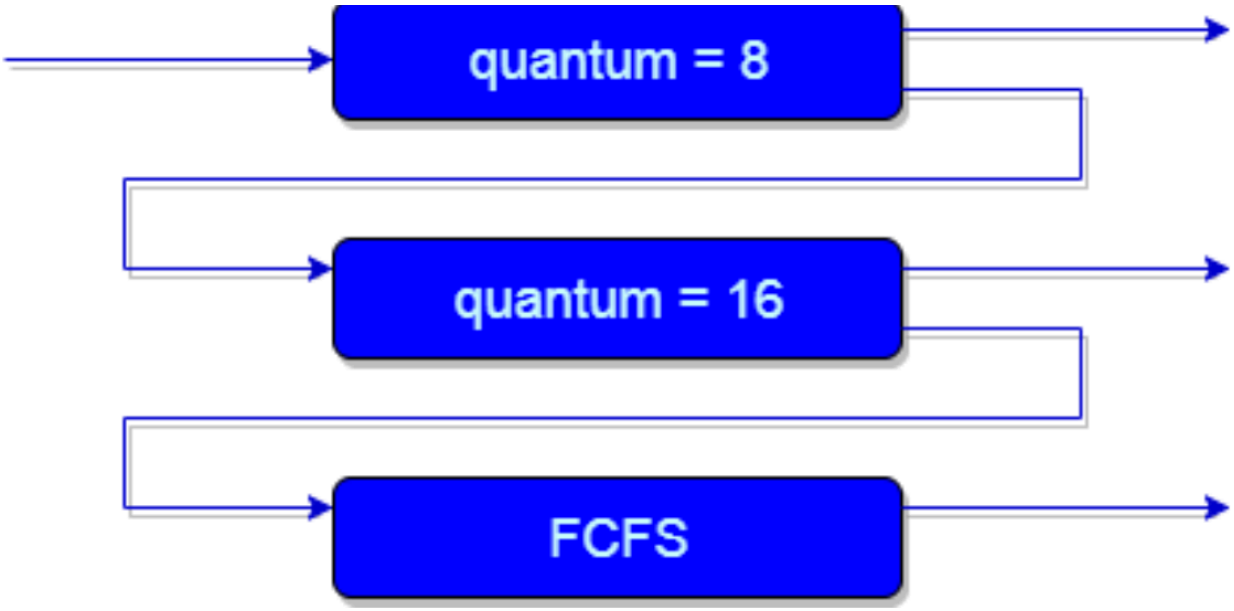

Figure 3: Processes of Multilevel Feedback Queue Scheduling(MLQ) (*StudyTonight 2022*)

Filesystem

The file system is how the operating system organizes files on the system's disk and a system that keeps or organizes files or materials on your computer so that they can be easily discovered and accessed. Also, it is managing physical storage, such as hard disks or CD-ROMs. A file system is a system that gives a name to a file and indicates where to place the file for storage and navigation. It also refers to the way files are organized on a disk. This is supported by almost all operating systems (OSs) as well as Linux. It is in a similar context to make it easy to search and navigate the library with filters such as authors, book classifications, and book titles. Unlike Windows, Linux uses an Extended File System (Ext) and focuses on ext2, ext3 and ext4 of the ext that is used the most among file systems. As shown below, there is a table of ext information.

Category	Ext1	Ext2	Ext3	Ext4
Developer	Stephen Tweedie	Rémy Card	Stephen Tweedie	Mingming Cao, Andreas Dilger, Alex Zhuravlev (Tomas), Dave Kleikamp, Theodore Ts'o, Eric Sandeen, Sam Naghshineh, others (Theodore Ts'o presented to make ext4
Date	04/1992	01/1993	11/2001	10/2006
Partition ID	-	Apple_UNIX_SVR2, 0x83(MBR), EBD0A0A2-B9E5-4433-87C0-68B6B72699C7 (GPT)	0x83(MBR), EBD0A0A2-B9E5-4433-87C0-68B6B72699C7(GPT)	s0x83(MBR), EBD0A0A2-B9E5-4433-87C0-68B6B72699C7 (GPT)
Directory Structure	-	Table	Table, Hashed B-tree(dir_index)	Linked List, Hashed B-tree
File Structure	Bitmap(free space), table(meta data)	Bitmap(free space), table(meta data)	Bitmap(free space), table(meta data)	Extents/Bitmap
Bad Block Structure	Table	Table	Table	Table
Maximum File size	-	16GB - 2TB	16GiB - 2TiB	16TiB (based on 4k blocks)
Maximum Number of File	-	10 to the 18th power	Variable, assigned at writing time	4 billion (saved at file system creation time)
Maximum File Name Length	-	255byte	255byte	256byte
Maximum Volume Size	-	2TB - 32TB	2TiB - 16TiB	1 EiB
Date Permissions	-	Modification(mtime), Characteristic Modification(ctime), Access(atime)	Modification(mtime), Characteristic Modification(ctime), Access(atime)	Modification(mtime), Characteristic Modification(ctime), Access(atime), Delete(dtime), Create(ctime)
Data Range	-	14/12/1901 ~ 18/01/2038	14/12/1901 ~ 18/01/2038	14/12/1901 ~ 25/04/2514
Data Accuracy	-	1s	1s	Nano second

File System Permissions	POSIX	POSIX	Unix permissions, ACLs, arbitrary security features (Linux 6.2 and later)	POSIX
Operating System	-	Linux, BSD, WINDOWS (through IFS), OSX(through IFS)	Linux, BSD, WINDOWS, OSX(through IFS)	Linux, WINDOWS (when using ext2fsd)
Characteristic	-	-	sNo-atime, append-only, synchronous-write, no-dump, h-tree(directory), immutable, journal, secure-delete, top(directory), allow-undelete	Extents, noextents, mballoc, nomballoc, delalloc, nodelalloc, data=journal, data=ordered, data=writeback, commit=nrsec, orlov, oldalloc, user_xattr, nouser_xattr, acl, noacl, bsddf, minixdf, bh, nobh, journal_dev

Table1: Summary of Ext at the file system

Ext2 is not supported for compression but is available through patches. Encryption is not supported. All files or directories are represented by inodes. The inode includes data on the size, rights, ownership, and location of the disk in the file or directory. In the inode, there is a pointer to the first 12 blocks containing data from the file. There are pointers to indirect blocks (including pointers to the next set of blocks), pointers to dual indirect blocks (including pointers to indirect blocks), and pointers to triple-indirect blocks (including pointers to double indirect blocks). E2compr is a variant of the ext2 file system that supports online compression and decompression in the Linux kernel. Ext3 does not support compression. Encryption is not supported (provided at the block device level). The maximum number of subdirectories that can be created in a subdirectory is 31,998. This is because the inode supports 32000 links. Journaling is divided into Journal, Ordered, and Writeback. Ext3 Advantages are data can be deleted from ext2 and changed to ext3 without a loss (no data backup required). Journalizing. Increase the online file system. Htree for large-scale directories (advanced version of B-tree). e2fsck disappears (e2fsck, which existed in ext2, is forced to scan all files after the system is down and is irrevocable). Ext3 Disadvantages are low processing speed compared to JFS, ReiserFS, XFS, etc. There is no fragmentation function available at the file system level. The checksum is not checked when recording in a journal. Ext4 eliminates the 64-bit memory space limit, improves the performance of ext3, and is a backward-compatible extension, developed by Cluster File Systems. However, other kernel developers have proposed to fork from ext3 and rename it to ext4 without affecting ext3 users. Compression is not supported, and encryption is not supported. Ext4 Advantages are large file system: supports up to 1 exabyte of volume and up to 16 terabytes of files. Extent: It is intended to replace the traditional block mapping methods used in ext2 and ext3. Backward compatibility: Backward compatibility for ext3 and ext2 allows the ext3 and ext2 file systems to be mounted as ext4. However, it is impossible to mount with ext3 if extents, an important new feature of ext4, are used. Delayed Allocation: It uses a file system performance technique called allocate-on-flush. 32000 subdirectories are unrestricted, ext4 has increased to 64000, and the dir_nlink feature allows for a larger number. Continuous performance can be obtained using the H-tree index. Ext4 Disadvantage is delayed allocation and potential data loss: Because

delayed allocation changes the behaviour intended by the programmer in ext3, it poses an additional risk of data loss in case of system crashes or power outages before all data is written to disk. More than kernel 2.6.30 automatically notice this and revert to the previous behaviour.

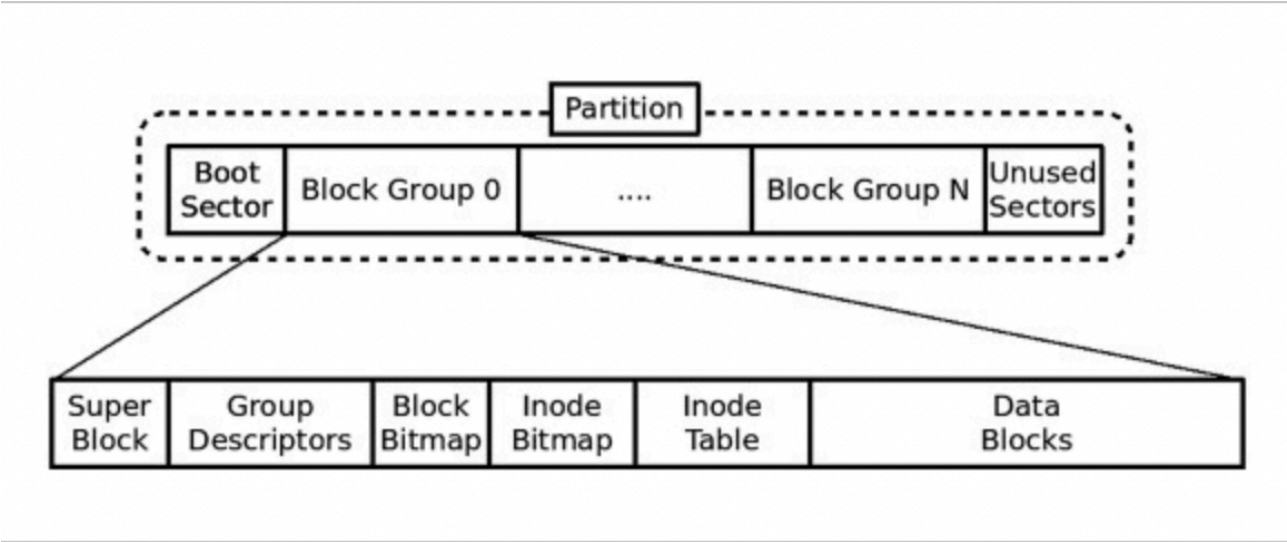

Figure4: The Structure of the EXT file system (*Broosen 2022*)

Memory Management

Any program must be loaded into memory to be processed and loaded into the processor. Here are some explanations that go into memory management about virtual memory of the Linux kernel. Virtual memory is a method of managing memory, which refers to a method of giving each program a virtual memory address rather than a real memory address. Using the virtual memory, the size of the actual physical memory can be logically expanded and used. Then why use virtual memory earlier?

-Logically, memory can be extended and used. It can be expanded to a larger space than the actual physical memory. This does not mean that data can be stored more than the actual physical memory has, but it can continuously map space such as RAM, disk, and register to virtual space.

-It can provide processes with the same memory space. Users only need to pay attention to the virtual memory space provided by the OS, not physical memory information. If only physical memory is used without virtual memory, both processes A and B are currently running simultaneously, the two processes must work concerning each other's physical memory space.

-Be efficient in memory management. It is fast to read and process continuously existing data. However, people act for generating, deleting, and modifying a lot of data while using a computer. Therefore, data that were initially allocated continuously are stored up and down over time. Using virtual memory, such non-continuous memory can be used in a logically continuous form of memory.

The unit of the virtual memory is managed on a page basis, and the physical memory is managed on a frame basis. The page and the frame have the same size. In order to use the memory efficiently, actual management is carried out using this paging technique. More internally, the pages are fixed sizes determined by the hardware. Conversely, segments are divided into logical units of programs such as method, procedure, function, object, variables, and stack. Unlike the paging method, which mechanically divides the page size into a constant size, the segmentation method is a method that divides the logical units of the program well and protects them from invading each other. Therefore, the actual memory is managed by mixing the paging technique and the segmentation technique. By using virtual memory, non-

continuous physical memory can be used as continuous memory. This meaning can be processed continuously using that page table.

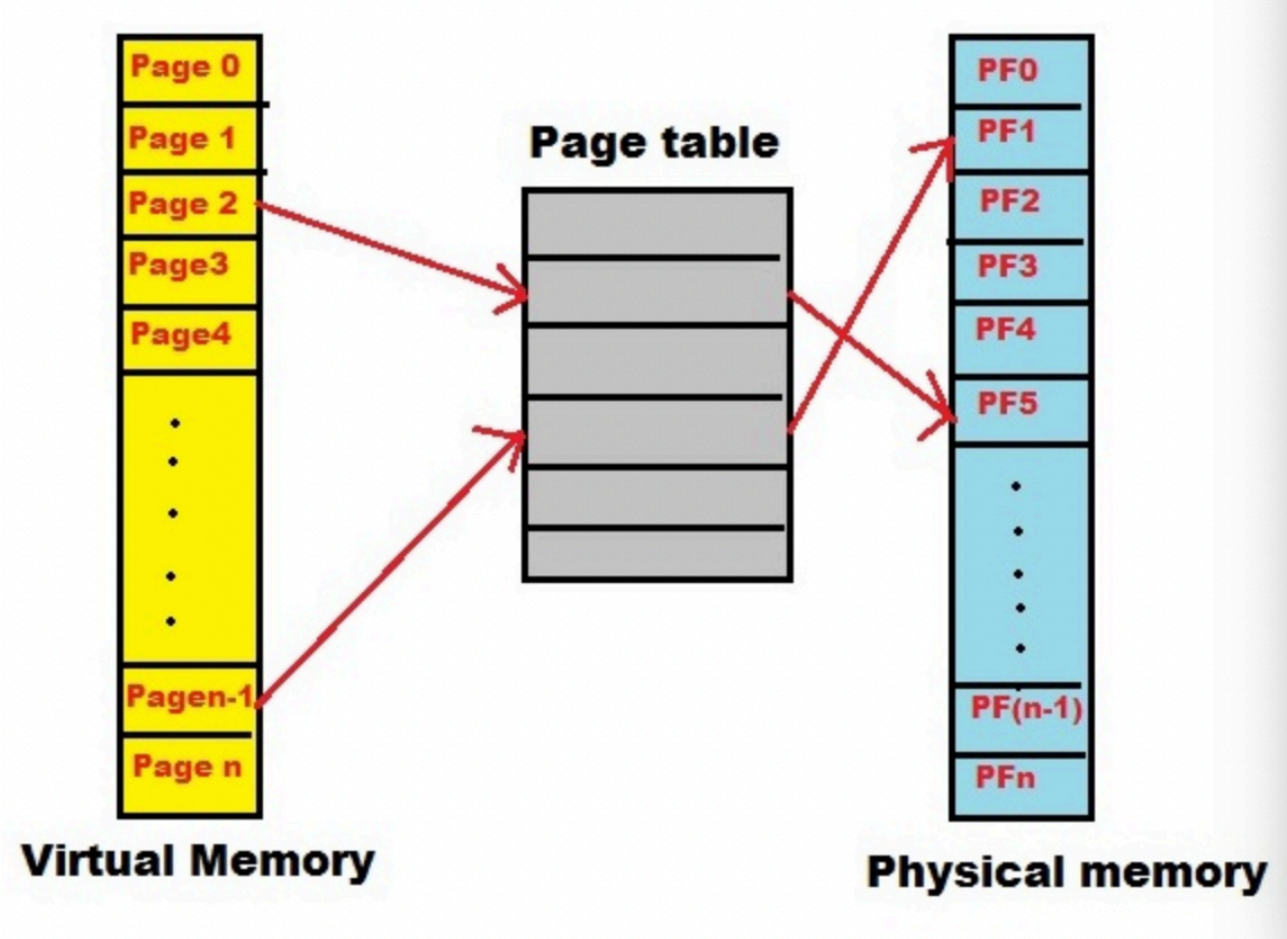

Figure 5: Linux Memory Management(Uncategorized 2017)

The process of converting an actual virtual address to a physical address is performed by hardware called a memory management unit (MMU). MMU works inside the actual CPU core. First, the Translation Lookaside Buffer (TLB) present in the MMU is referred to. TLB contains information about the most recently converted page table entry. The MMU first searches the TLB for the virtual address to be converted through the CPU. If there is an entry that is uploaded, you can immediately change your address to physical memory and get the data you want.

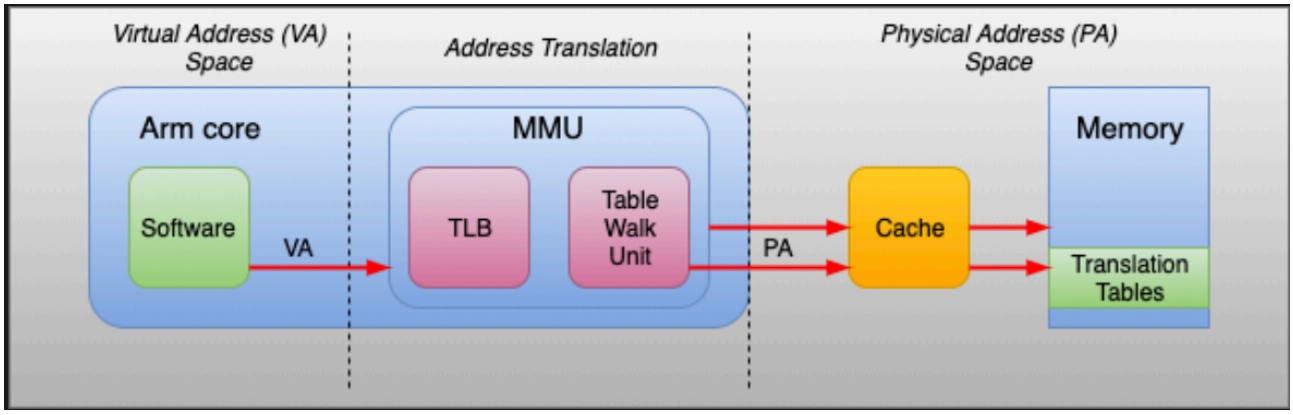

Figure 6: Memory Management Unit (MMU) Overview (Uncategorized 2017)

The good things about virtual memory are that virtual memory assists speed up when only a specific segment of the program is needed for execution of the program, this is very good in implementing a multiprogramming environment, people are able to execute more applications at once, it assists to input many big programs into small ones, and more processes must be maintained in the main memory, which improves the efficient use of the CPU (*Williams 2022)*. The bad things about virtual memory are that If the system works with virtual memory, the application can run slowly, switching between applications can take longer, gives less hard drive space, decreases system reliability, it does not give the same performance as RAM, and it negatively impacts the overall performance of the system (*Williams 2022*).

Custom OS design

One important principle is to separate policies from mechanisms. The mechanism is to decide what to do and how to do it, and the policy is to decide what to do. For example, the interrupt generation structure through timers is a mechanism to ensure CPU protection, but it is a policy decision to determine how long the timer should be set for a particular user. The separation of policy and mechanisms is very important for flexibility. Policies can change places or over time. In the worst case, the policy change requires a change in the underlying mechanism. A general mechanism that is not sensitive to changes in policy is more preferable. This is because it is clear that maintenance and repair will be easier from the developer's point of view. In summary, if it is policy to decide what to do, the mechanism is to decide how to do it. One of the most commonly used operating systems for machine learning is Linux and the open-source nature of the Linux environment is well adapted to the complex installation and configuration processes required for many machine learning applications. It uses a learning rate scheduler. As learning progresses, a lower loss value can be obtained if the learning rate can be changed according to the situation. This requires a learning rate schedule.

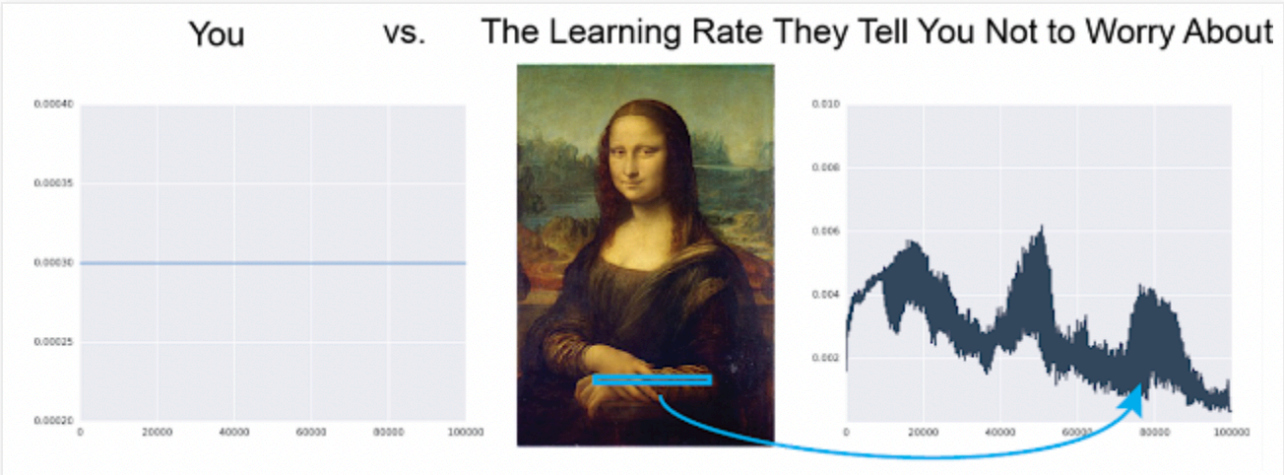

Figure 7: The Learning Rate Schedule(*Jang, E, Raffel, C, Gulrajani, I, Almeida, D 2018*)

The learning rate schedule is a significant hyperparameter to select when training neural nets. Adjust the learning rate too high, and the loss function can fail to converge. Adjust the learning rate too low, and the model can take a long time to train, or even worse, overfit (*Jang, E, Raffel, C, Gulrajani, I, Almeida, D 2018*). It also uses different learning rates for each layer, and it is recommended to set weights so that each layer has a similar weight update rate. Like logistic functions, learning output is important when using finite-valued activation functions, and the learning rate is small for layers close to the output layer and large for layers close to the input layer. Also, it uses the Cluster File System. A cluster file system is a scale-out

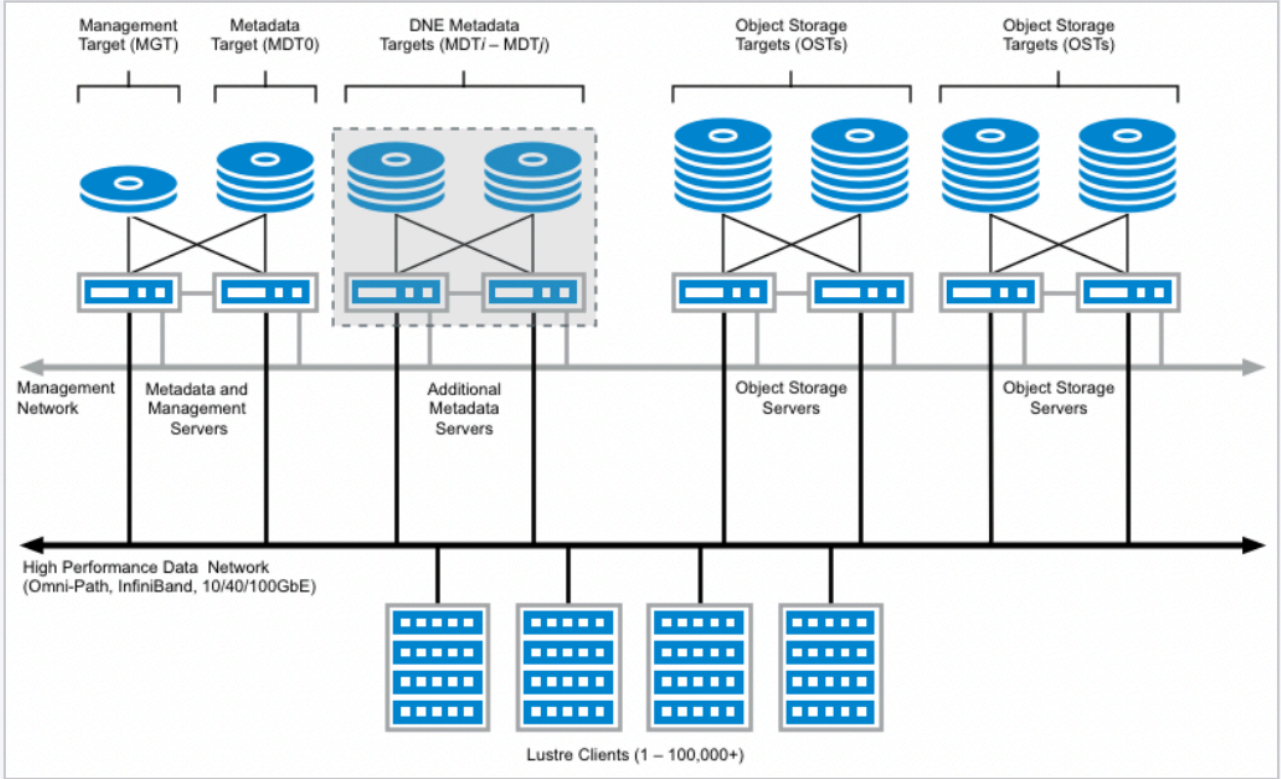

Figure 8: Cluster Architecture(Wiki 2022)

a distributed file system that can be used by client hosts to connect to capacity. Simple architectural configuration, safety, high-capacity I/O throughput and application are superior compared to traditional distributed file systems. The cluster file system does not have a metadata server. This is because metadata information for all files is not needed. All servers know only the servers in the cluster storage pool and the volumes that used them. As a result, all servers in a cluster file system cluster can get information about the volumes that are currently mountable. The cluster file system server has its local file system. When saving files to a cluster file system, the files sent by the client are distributed to the specified folder on the local disk of the cluster file system server. In addition, once the operating system is designed, it will have to be implemented using several methods. It is recommended to use virtual memory management for memory management because it requires a lot of memory and can be expensive and difficult to handle when handled by physical devices.

References

StudyTonight (2022) *CPU Scheduling in Operating System*, viewed 08 June 2022, <https://www.studytonight.com/operating-system/cpu-scheduling>.

StudyTonight (2022) *Multilevel Queue Scheduling Algorithm*, 08 June 2022, <https://www.studytonight.com/operating-system/multilevel-queue-scheduling>.

The Coordinator (2018) *A CPU Scheduler Simulator*, CobWeb, viewed 08 June 2022, <http://cobweb.cs.uga.edu/~maria/classes/x730-Spring-2018/project4-scheduler.html>.

StudyTonight (2022) *Multilevel Feedback Queue Scheduling*, viewed 09 June 2022, <https://www.studytonight.com/operating-system/multilevel-feedback-queue-scheduling>.

GeeksforGeeks (2022) *CPU Scheduling in Operating Systems*, viewed 09 June 2022, <https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/>.

White, S (2019) *CPU Scheduling: What it is and its roles?*, Allassignmenthelp, viewed 09 June 2022, <https://www.allassignmenthelp.com/blog/cpu-scheduling-what-it-is-and-its-roles/>.

Wikipedia (2022) *Extended File System*, viewed 10 June 2022, <https://en.wikipedia.org/wiki/File_system>.

Wikipedia (2022) *ext2*, viewed 10 June 2022, <https://en.wikipedia.org/wiki/Ext2>.

Wikipedia (2022) *ext3*, viewed 10 June 2022, <https://en.wikipedia.org/wiki/Ext3>.

Wikipedia (2022) *ext4*, viewed 10 June 2022, <https://en.wikipedia.org/wiki/Ext4>.

Broosen, D (2022) *what is Ext4 (Ext2, Ext3) – Linux File System*, Recoverhdd, viewed 10 June 2022, <https://recoverhdd.com/blog/the-ext-ext2-ext3-ext4-filesystem.html>.

Uncategorized (2017) *Memory Management*, Tutorialsdaddy, viewed 10 June 202, <https://www.tutorialsdaddy.com/uncategorized/memory-managment/>.

Pearson, G (2020) *DEEP DIVE: MMU VIRTUALIZATION WITH XEN ON ARM*, Starlab, viewed 10 June 2022, <https://www.starlab.io/blog/deep-dive-mmu-virtualization-with-xen-on-arm>.

Williams, L (2022) *Virtual Memory in OS: What is, Demand Paging, Advantages*, Guru99, viewed 10 June 2022, <https://www.guru99.com/virtual-memory-in-operating-system.html#8>.

Kordek, A (2021) *Machine Learning and Operating Systems*, InmotionHosting, viewed 10 June 2022, <https://www.inmotionhosting.com/support/product-guides/private-cloud/machine-learning-and-operating-systems/>.

Jang, E, Raffel, C, Gulrajani, I, Almeida, D (2018) *Aesthetically Pleasing Learning Rates*, Evjang, viewed 10 June 2022, <https://blog.evjang.com/2018/04/aesthetic-lr.html>.

Wiki (2022) *Introduction to Lustre*, viewed 10 June 2022, <https://wiki.lustre.org/Introduction_to_Lustre>.

Travis (2022) *What Is Cluster File System In Linux?*, Systranbox, viewed 10 June 2022, <https://www.systranbox.com/what-is-cluster-file-system-in-linux/>